

Agenda

- What is OpenTelemetry?
 - A Brief History
- Distributed Tracing Overview
- OpenTelemetry
 - Instrumentation
 - Collectors
- Q&A
- Put it in action

What is OpenTelemetry?



OpenTelemetry is a set of APIs, SDKs, tooling and integrations that are designed for the creation and management of *telemetry data*, such as traces, metrics, and logs.

OpenTelemetry's Mission is to enable effective observability by making high-quality, portable telemetry ubiquitous and vendor-agnostic.



A trace represents a single user's journey across multiple applications and systems (usually microservices).

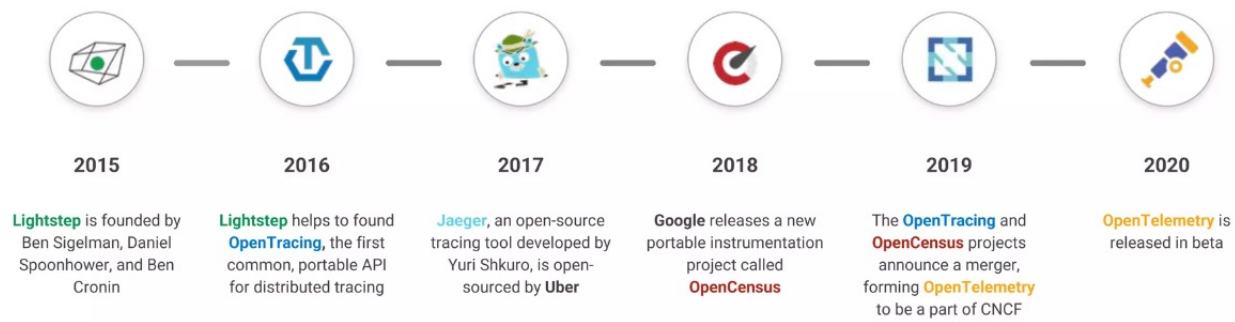


Numeric data measured at various time intervals (time series data); SLI's (request rate, error rate, duration, CPU%, etc.)



Timestamped records of discrete events that happened within an application or system, such as a failure, an error, or a state transformation

OpenTelemetry - A brief history



© 2022 Lightstep from ServiceNow, Inc. All Rights Reserved.



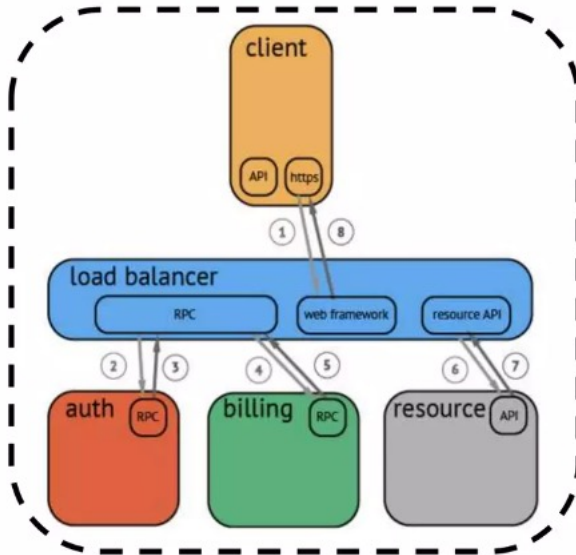
Distributed systems - a brief history



What are
distributed traces?

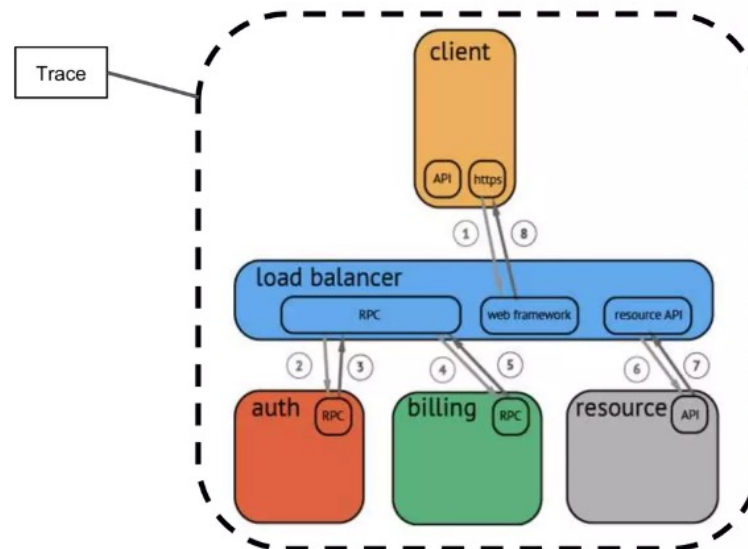


Introduction to distributed traces



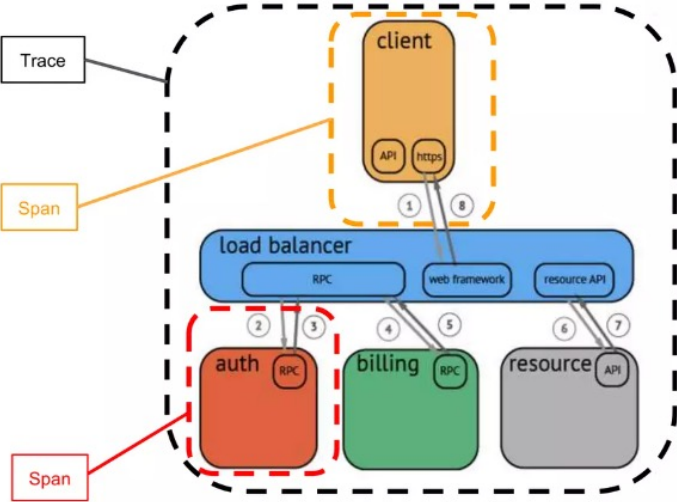
A distributed trace provides a [view of the life of a request](#) as it travels across multiple hosts and services communicating over various protocols

Introduction to distributed traces



A “**trace**” is a view into the request lifecycle as a whole as it moves through a distributed system.

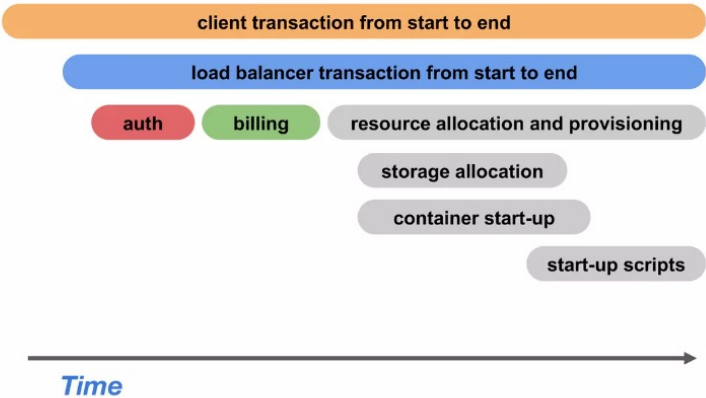
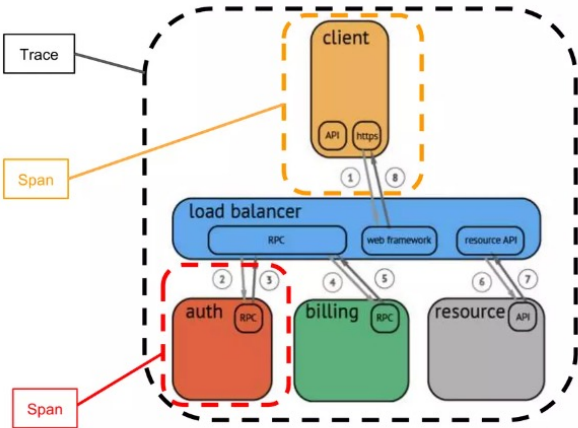
Introduction to distributed traces



A **trace** is a collection of spans.

Each component of the distributed system contributes a “**span**” - a named, timed operation representing a piece of the workflow.

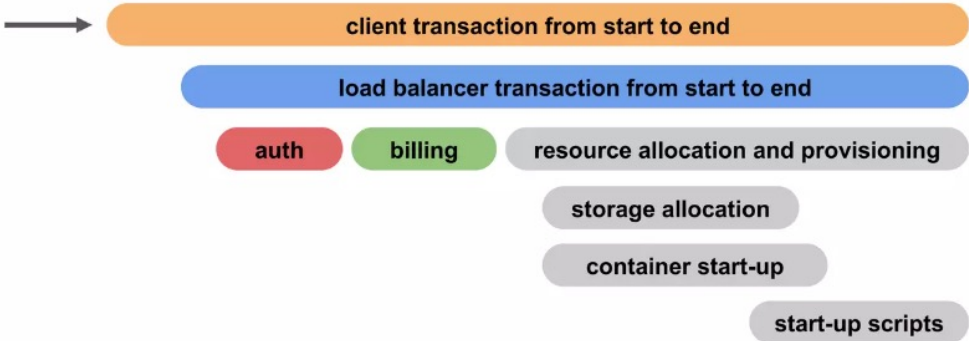
Introduction to distributed traces



Introduction to distributed traces



A trace can be visualized as a tree of spans with a “**root**” span at the top.

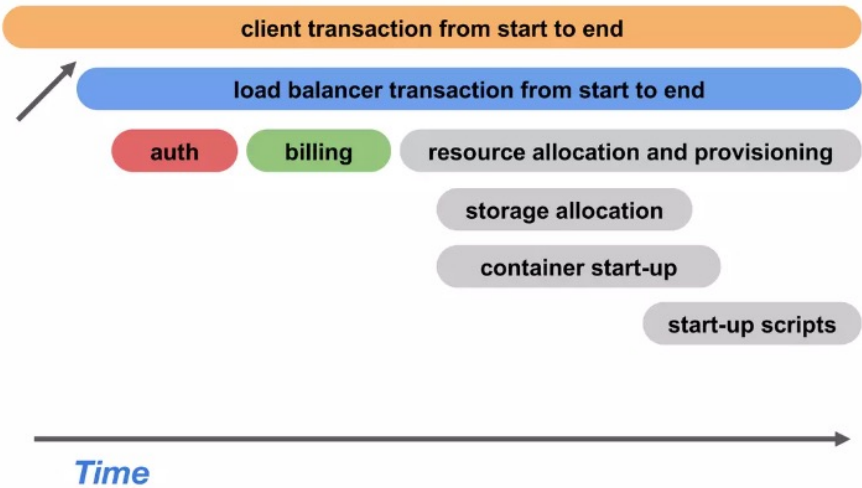


Introduction to distributed traces



Spans have **parent-child** relationships.

Parent spans can have multiple children.



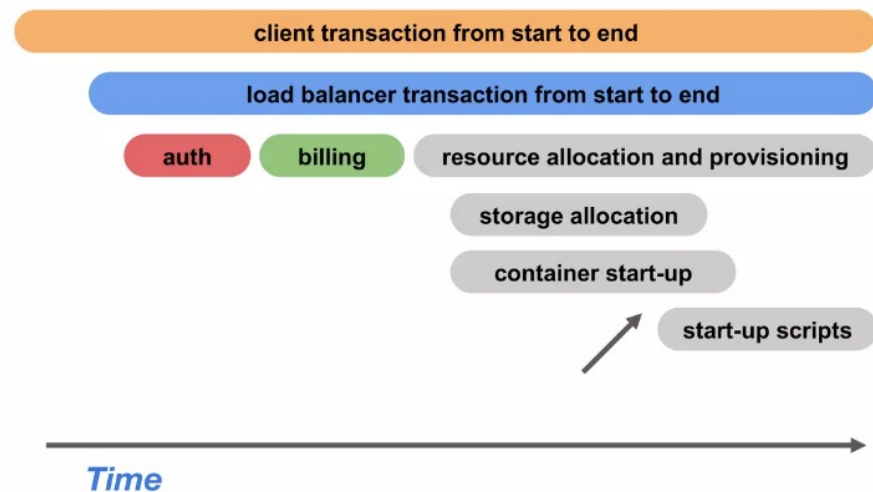
Introduction to distributed traces



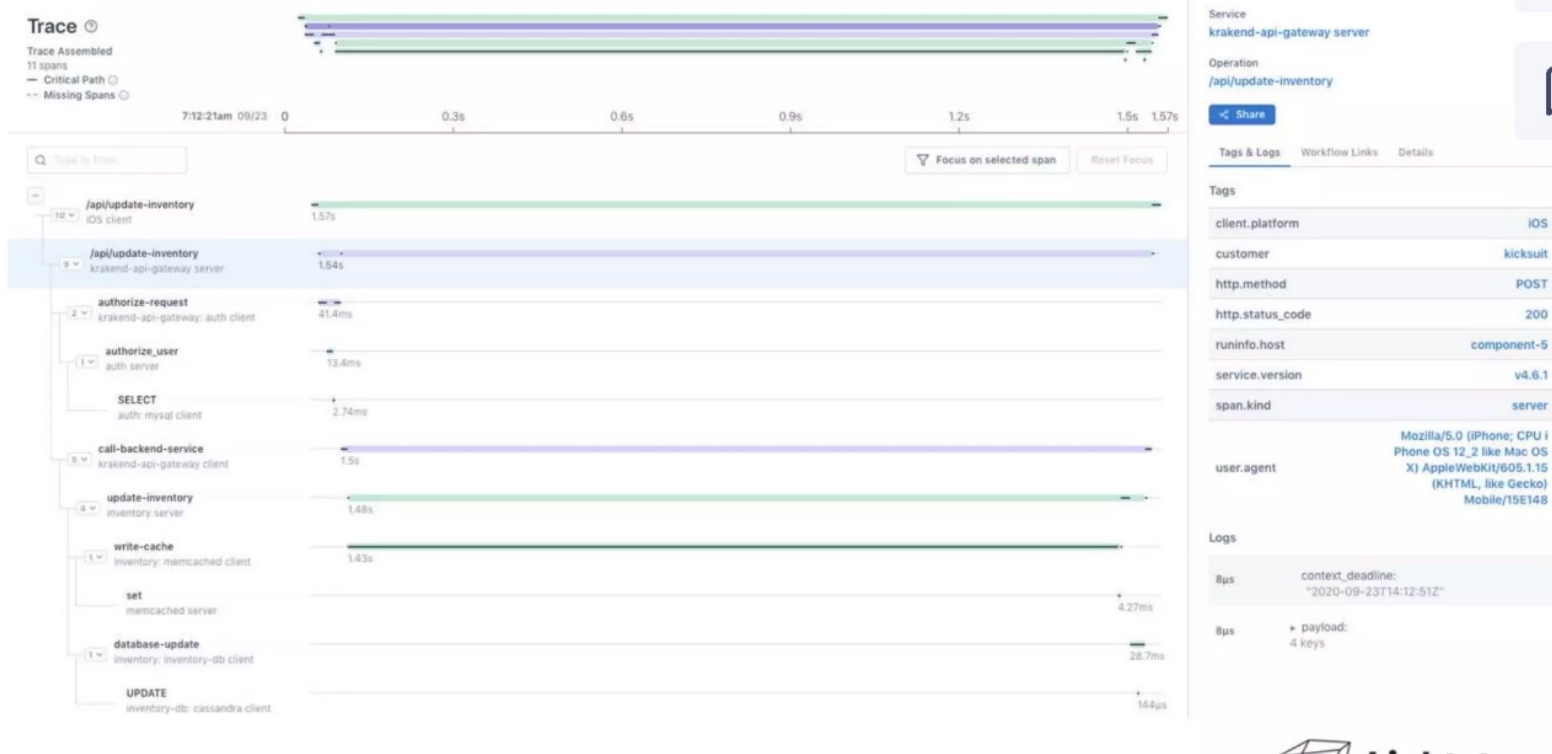
Child spans don't necessarily finish before their parents

This can be seen with asynchronous children

Real world example: An rpc call timeout
out - the parent span finished earlier
than the hanging child



Visualizing a trace



Visualizing a trace

Each span also has **span context**

Span context is composed of **attributes (tags)** and **events (logs)**. These are added during instrumentation.

Attributes allow you to search and segment your data within LightStep

Events (logs) add information that is useful during root cause and debugging

The screenshot displays a trace for the service 'krakend-api-gateway server' and operation '/api/update-inventory'. It includes a 'Share' button and tabs for 'Tags & Logs', 'Workflow Links', and 'Details'. The 'Tags & Logs' tab is active, showing a list of tags and a log entry.

Tags	
client.platform	ios
customer	kicksuit
http.method	POST
http.status_code	200
runinfo.host	component-5
service.version	v4.6.1
span.kind	server
user.agent	Mozilla/5.0 (iPhone; CPU i Phone OS 12_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148

Logs

Timestamp	Event
8µs	context_deadline: "2020-09-23T14:12:51Z"
8µs	payload: 4 keys

OpenTelemetry Instrumentation

OpenTelemetry support of dev languages



LANGUAGE	TRACE STATUS*	INSTRUMENTATION MANUAL/AUTO**
C++	stable	manual
C# / .NET	stable	manual & auto
Erlang / Elixir	stable	manual
Go	stable	manual
Java	stable	manual & auto
Javascript / Node	stable	manual & auto
Javascript / Browser	stable	manual & auto

You want to know more?

See <https://opentelemetry.io/docs/instrumentation/>

LANGUAGE	TRACE STATUS*	INSTRUMENTATION MANUAL/AUTO**
PHP	pre-alpha	manual
Python	stable	manual & auto
Ruby	stable	manual & auto
Rust	beta	manual
Swift	beta	manual

(*) Trace implementation status as of end of April 2022

(**) Automatic instrumentation means quick wins as no need to update existing code to collect some data

Supported languages versions:

- .NET & .NET Framework all supported versions except .NET Fwk v3.5 as <https://github.com/open-telemetry/opentelemetry-dotnet>
- Java > v1.8
- NodeJS >v10 as <https://github.com/open-telemetry/opentelemetry-js>
- Python, only latest versions as of <https://github.com/open-telemetry/opentelemetry-python>

OTel Instrumentation Steps

- Configure Tracer
 - Decorate Spans
- Context Propagation

Configure Tracer

```
const opentelemetry = require('@opentelemetry/api');
const { NodeTracerProvider } = require('@opentelemetry/node');
const { SimpleSpanProcessor } = require('@opentelemetry/tracing');
const { CollectorTraceExporter } = require('@opentelemetry/exporter-collector');

const collectorOptions = {
  serviceName: '<Service Name>', // Provide a service name
  url: 'https://<Satellite Host>:<Satellite Port>/api/v2/otel/trace', // URL and port to Lightstep Satellite
  headers: {
    'Lightstep-Access-Token': 'YOUR_TOKEN' //Lightstep Access Token
  },
};

// Create an exporter for sending span data
const exporter = new CollectorTraceExporter(collectorOptions);
// Create a provider for activating and tracking spans
const tracerProvider = new NodeTracerProvider({
  plugins: {
    http: {
      enabled: true,
      // You may use a package name or absolute path to the file.
      path: '@opentelemetry/plugin-http',
      // Enable HTTP Plugin to automatically inject/extract context into HTTP headers
    }
  }
});

// Configure a span processor for the tracer
tracerProvider.addSpanProcessor(new SimpleSpanProcessor(exporter));
// Register the tracer
tracerProvider.register();
```

← Declare Dependencies

← Declare Destination Options

← Declare Plugins



Send Telemetry To OpenTelemetry Collector

```
...  
const collectorOptions = {  
  serviceName: '<Service Name>', // Provide a service name  
  url: 'https://<Collector Host>:<Collector Port>', // URL and port to OTel Collector  
};  
  
const exporter = new CollectorTraceExporter(collectorOptions);  
...
```

Only difference between sending traffic to an OTel Collector and a Lightstep Satellite is the address and Access Token

Create Spans

```
const tracer = opentelemetry.trace.getTracer();
```

```
const span = tracer.startSpan('foo');  
span.setAttribute('platform', 'osx');  
span.setAttribute('version', '1.2.3');  
span.addEvent('event in foo');
```

```
const childSpan = tracer.startSpan('bar', {  
  parent: span,  
});
```

```
childSpan.end();  
span.end();
```

Start Span

Decorate Span

Create Child Span

End Span

Tracer Span Methods

- `tracer.startSpan(name, {parent: <span>, ...})`
 - This method returns a child of the specified span.
- `api.context.with(api.setSpan(api.context.active(), name))`
 - Starts a new span, sets it to be active. Optionally, can get a reference to the span.
- `api.trace.getSpan(api.context.active())`
 - Used to access & add information to the current span
- `span.addEvent(msg)`
 - Adds structured annotations (e.g. "logs") about what is currently happening.
- `span.setAttributes(core.Key(key).String(value)...)`
 - Adds a structured, typed attribute to the current span. This may include a user id, a build id, a user-agent, etc.
- `span.end()`
 - Often used with `defer`, fires when the unit of work is complete and the span can be sent



Context Propagation

- Distributed context is an abstract data type that represents collection of entries.
- Each key is associated with exactly one value.
- It is serialized for propagation across process boundaries
- Passing around the context enables related spans to be associated with a single trace.
- W3C TraceContext is the de-facto
- Context propagation can be automatic
- Manual propagation code can be found

telemetry.github.io/opentelemetry-

Putting It All Together

```
const opentelemetry = require('@opentelemetry/api');
const { NodeTracerProvider } = require('@opentelemetry/node');
const { SimpleSpanProcessor } = require('@opentelemetry/tracing');
const { CollectorTraceExporter } = require('@opentelemetry/exporter-collector');

const collectorOptions = {
  serviceName: '<Service Name>', // Provide a service name
  url: 'https://<Satellite Host>:<Satellite Port>/api/v2/otel/trace', // URL and port to
  headers: {
    'Lightstep-Access-Token': 'YOUR_TOKEN' //Lightstep Access Token
  },
};

// Create an exporter for sending span data
const exporter = new CollectorTraceExporter(collectorOptions);
// Create a provider for activating and tracking spans
const tracerProvider = new NodeTracerProvider({
  plugins: {
    http: {
      enabled: true,
      // You may use a package name or absolute path to the file.
      path: '@opentelemetry/plugin-http',
      // Enable HTTP Plugin to automatically inject/extract context into HTTP headers
    }
  }
});
```

Continued....

```
// Configure a span processor for the tracer
tracerProvider.addSpanProcessor(new
SimpleSpanProcessor(exporter));
// Register the tracer
tracerProvider.register();

const tracer = opentelemetry.trace.getTracer();

const span = tracer.startSpan('foo');
span.setAttribute('platform', 'osx');
span.setAttribute('version', '1.2.3');
span.addEvent('event in foo');

const childSpan = tracer.startSpan('bar', {
  parent: span,
});

childSpan.end();
span.end();
```

SDKs, Exporters, and Collector Services

- OpenTelemetry's **SDK** implements trace & span creation.
- An **exporter** can be instantiated to send the data collected by OpenTelemetry to the backend of your choice.
 - E.g. Jaeger, Lightstep, etc.
- OpenTelemetry **collector** proxies data between instrumented code and backend service(s). The exporters can be reconfigured without changing instrumented code.



OpenTelemetry Collectors

What is the OpenTelemetry Collector?

The OTel Collector is an independent binary process, written in Go, that acts as a 'universal agent' for the OpenTelemetry ecosystem.

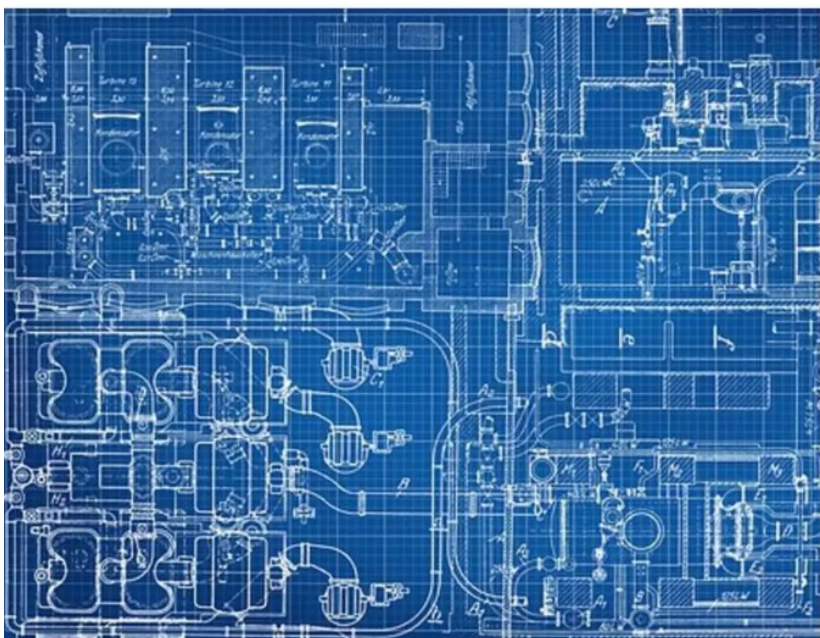
At a high level, the goals of the collector are...

- Collect, process, and export telemetry data in a highly performant and stable manner.
- Support multiple types of telemetry.
- Highly extensible and customizable, but with an easy OOB experience.

The collector provides separation of concerns between Developers and Operations/SRE teams.



Architecture of the Collector



The collector has 4 major components -

- Receivers
- Processors
- Exporters
- Extensions

Collectors receive, process, and forward telemetry to export destinations.

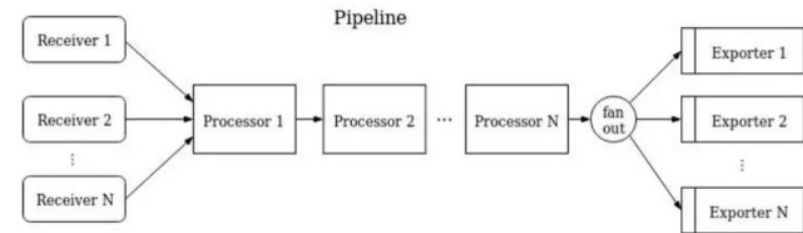
These can be part of one, or more, *pipelines*.



Pipelines

Telemetry data flows through the collector, from receiver to exporter. Multiple pipelines in a single collector are configurable, but each must operate on a single *telemetry type* - either traces, or metrics.

All data received by a pipeline will be processed by all processors and exported to all exporters.



YAML

```
service:
  pipelines: # section that can contain multiple subsections, one per pipeline
    traces: # type of the pipeline
      receivers: [opencensus, jaeger, zipkin]
      processors: [tags, tail_sampling, batch, queued_retry]
      exporters: [opencensus, jaeger, stackdriver, zipkin]
```

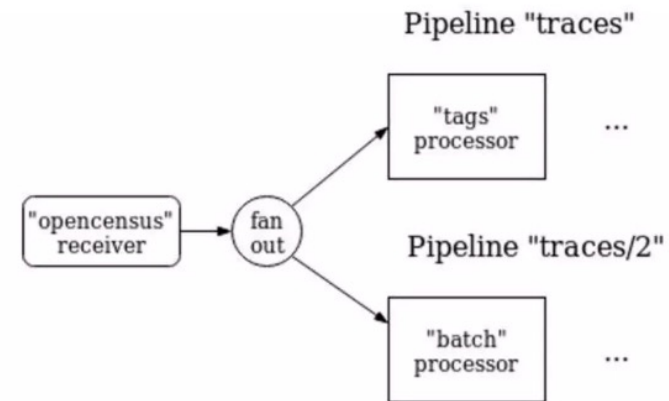
Receivers



Receivers listen on a single port for telemetry data. A single receiver can send to multiple pipelines.

Currently, the collector supports the following receivers -

- OTLP
- OpenCensus
- Fluentd Forward
- Jaeger
- Zipkin
- Prometheus
- Host Metrics



YAML

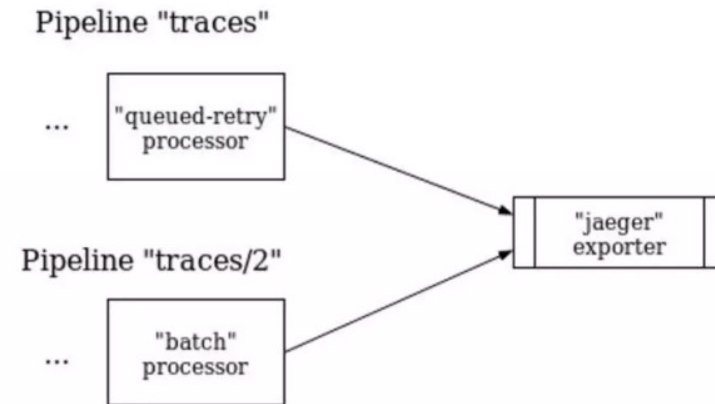
```
Receivers:
  otlp:
    grpc:
      endpoint: "0.0.0.0:55678"

service:
  pipelines:
    traces: # a pipeline of "traces" type
      receivers: [otlp]
      processors: [tags, tail_sampling, batch, queued_retry]
      exporters: [otlp]
```

Exporters

Typically, an exporter will forward all data it receives to a network endpoint, but can also write to a file or console.

Similar to receivers, a single exporter may be included in multiple pipelines.



YAML

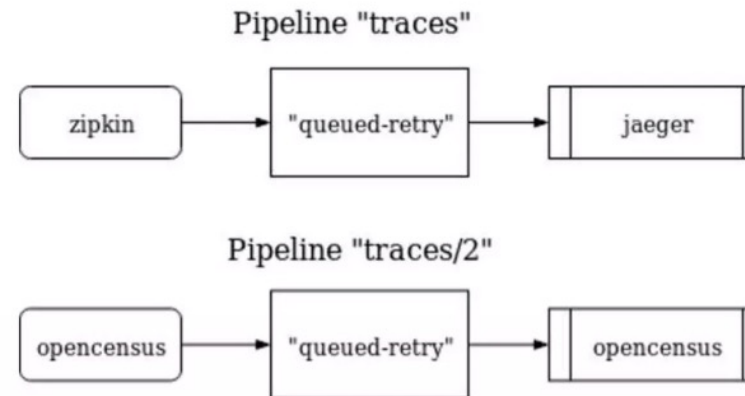
```
exporters:
  jaeger:
    protocols:
      grpc:
        endpoint: "0.0.0.0:14250"
    otlp:
      endpoint: <YOUR_LOAD_BALANCER_DNS_NAME_OR_IP_ADDRESS>
      headers: {"lightstep-access-token": "<YOUR_ACCESS_TOKEN>"}
service:
  pipelines:
    traces: # a pipeline of "traces" type
      receivers: [OTLP]
      processors: [tags, tail_sampling, batch, queued_retry]
      exporters: [jaeger, otlp]
```

Processors

Processors are very powerful!

They run sequentially, and receive all data from the processor before them. A processor can transform telemetry data, delete it, etc.

A single processor configuration can apply to multiple pipelines, but each pipeline gets a unique instance of the processor.



YAML

```
processors:
  queued_retry:
    size: 50
    per-exporter: true
    enabled: true
service:
  pipelines:
    traces: # a pipeline of "traces" type
      receivers: [zipkin]
      processors: [queued_retry]
      exporters: [jaeger]
    traces/2: # another pipeline of "traces" type
      receivers: [opencensus]
      processors: [queued_retry]
      exporters: [opencensus]
```



Processors Deep Dive

Receivers, Exporters, and Extensions aren't super interesting on the face of things - they receive or emit data, or simply give you some introspection on the collector instance.

Processors, however, can do a *lot*. Let's talk about them.

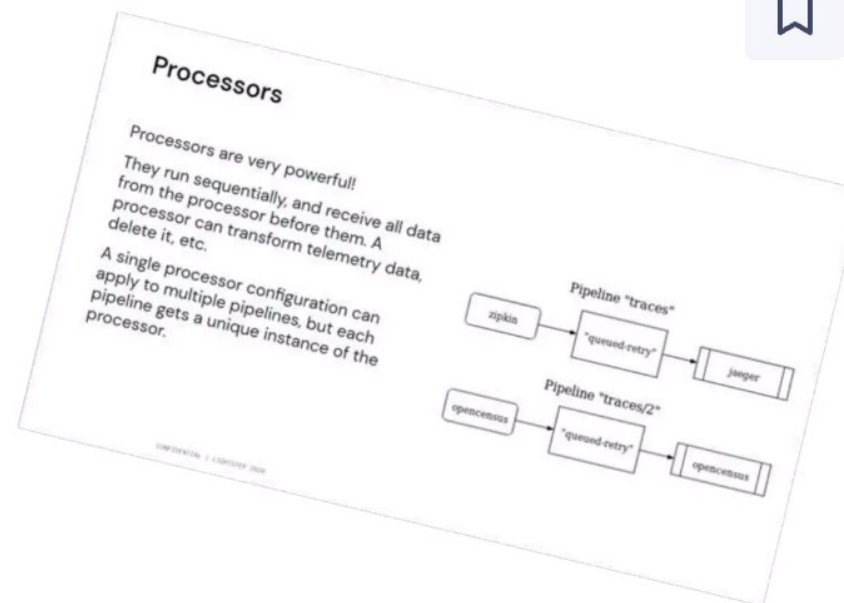


Processors Ordering

Remember this?

Processors one run after the other, so the order that you define them matters.

This means that limiting and sampling should be first, and batching/transforms should be later.



Memory Limiter

The memory limiter is used to prevent a collector from going OOM.

It does this by allowing the user to configure and tune GC on the collector.

This is not a replacement for sizing/tuning collector memory/cpu.

```
processors:
```

```
  memory_limiter:
```

```
    ballast_size_mib: 2000
```

```
    check_interval: 5s
```

```
    limit_mib: 4000
```

```
    spike_limit_mib: 500
```

Sampling Processors

Out of the box, the collector supports a *probabilistic sampler* and a *tail-based sampler*.

Probabilistic supports OpenTracing priority sampling hints as well as trace ID based hashing.

Tail sampling allows for user-defined policies such as string/number attributes or desired rate.



```
processors:
  probabilistic_sampler:
    hash_seed: 22
    sampling_percentage: 15.3
  tail_sampling:
    decision_wait: 10s
    num_traces: 100
    expected_new_traces_per_sec: 10
    policies:
    [
      {
        name: test-policy-1,
        type: always_sample
      },
      {
        name: test-policy-2,
        type: numeric_attribute,
        numeric_attribute: {key: key1, min_value: 50, max_value: 100}
      },
      {
        name: test-policy-3,
        type: string_attribute,
        string_attribute: {key: key2, values: [value1, value2]}
      },
      {
        name: test-policy-4,
        type: rate_limiting,
        rate_limiting: {spans_per_second: 35}
      }
    ]
]
```

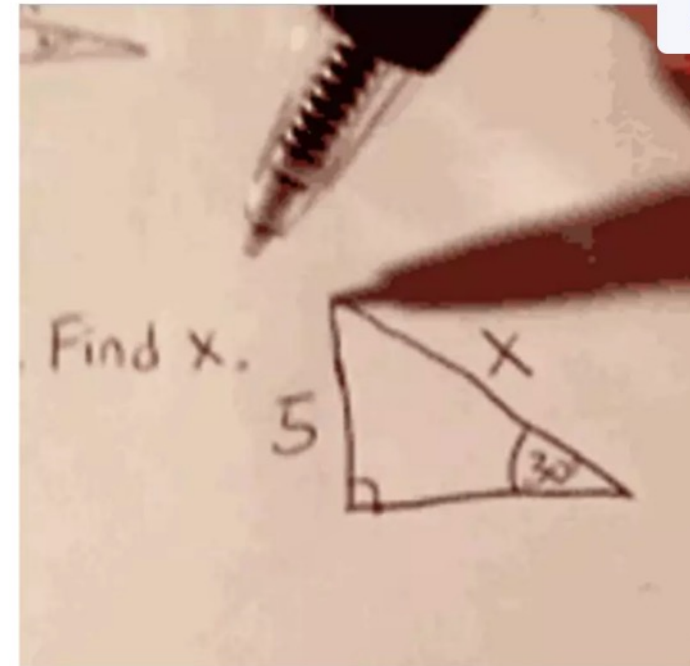
Attribute and Span Processors

Attribute processors can modify the attributes of a span; insert, update, upsert, delete, and extract based on attribute names or regular expressions.

Span processors can modify a span name or attributes based on a span name.

This can be used for more precise data scrubbing, creating attributes from span names, hashing attribute fields, and more.

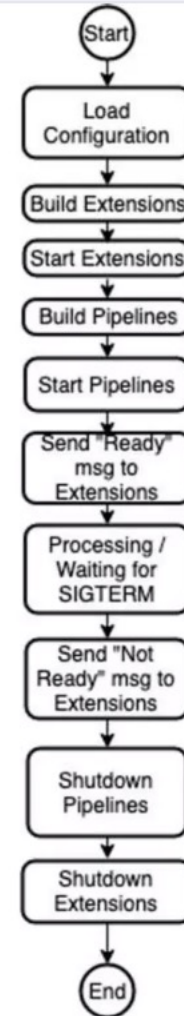
Note: Resource attributes are modified with the Resource processor.



Extensions

Functionality can be added to the collector through Extensions.

Currently, this is used for health checks and diagnostic information.



```
# Example of the extensions available with the core Collector. The list below
# includes all configurable options and their respective default value.
extensions:
  health_check:
    port: 13133
  pprof:
    endpoint: "localhost:1777"
    block_profile_fraction: 0
    mutex_profile_fraction: 0
  zpages:
    endpoint: "localhost:55679"

# The service lists extensions not directly related to data pipelines, but used
# by the service.
service:
  # extensions lists the extensions added to the service. They are started
  # in the order presented below and stopped in the reverse order.
  extensions: [health_check, pprof, zpages]
```



```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: "0.0.0.0:55680"
      http:
        endpoint: "0.0.0.0:55681"
processors:
  batch:
  queued_retry:
extensions:
  health_check: {}
exporters:
  otlp:
    endpoint: "ingest.lightstep.com:443"
    headers:
      "lightstep-access-token": "{{ .Values.lightstepKey }}"
service:
  extensions: [health_check, zpages]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch, queued_retry]
      exporters: [otlp]
```

This can be any receiver, not just OTLP.
Remember, receivers can be part of multiple pipelines!

Batch and Retry processors are recommended
to reduce connections and decrease size of
outgoing reports.

Other export options include
compression (gzip), etc. If satellites are
in Single Project mode, no headers
needed, just endpoint!

Putting It All Together

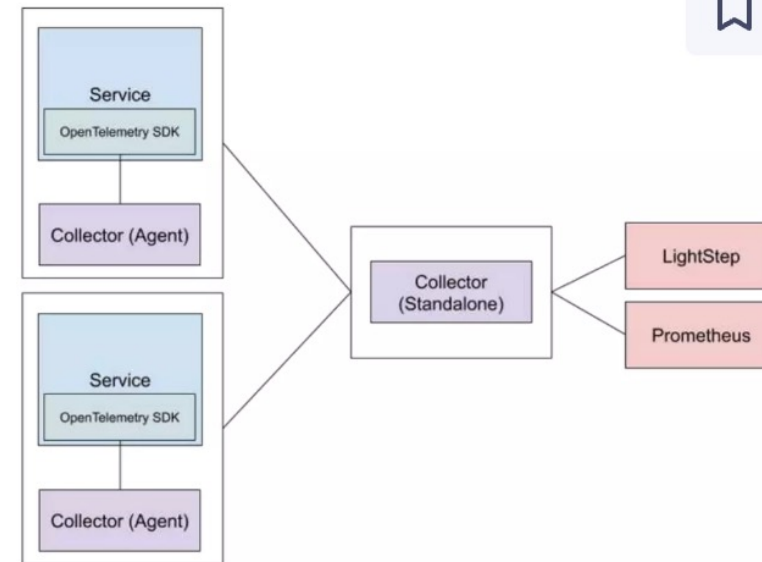


Collector Deployment Strategies

A goal of the collector is '5 Minutes to Value', and part of this is easy support for a variety of deployment strategies.

The collector can be run as an 'agent' or as a standalone (or pool of standalone) processes.

Eventually, the collector will support distribution packages for popular targets (rpm, docker, windows...) and automatic ingest of environment metrics/tags (AWS env resources, k8s, etc.)



Observability (n)



- *the ability to navigate **from effect to cause***



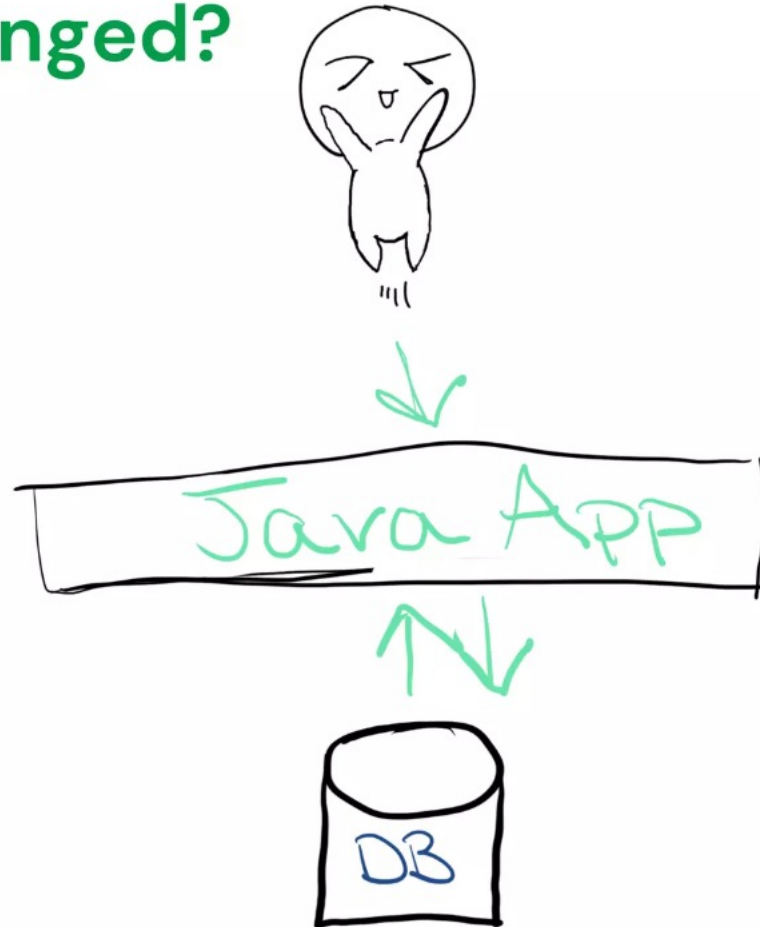
For example...

Spike in errors → misconfiguration

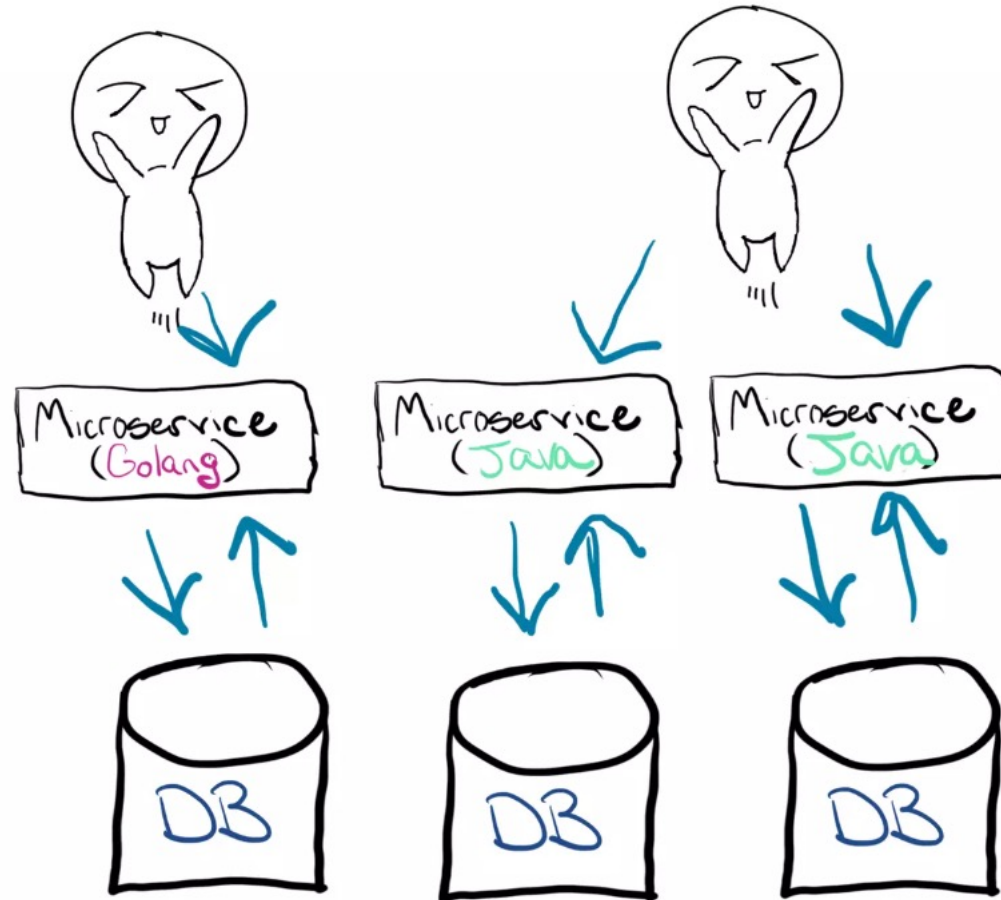
Increased latency → new customer behavior

User complaints → upstream service deployed

What Changed?



SCALE!

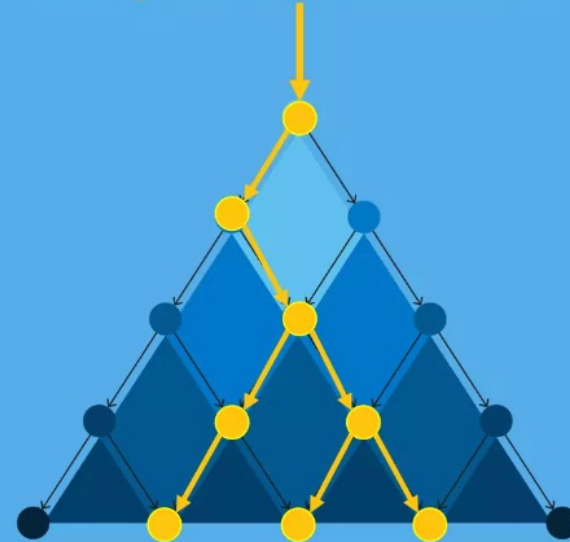


Context Propagation

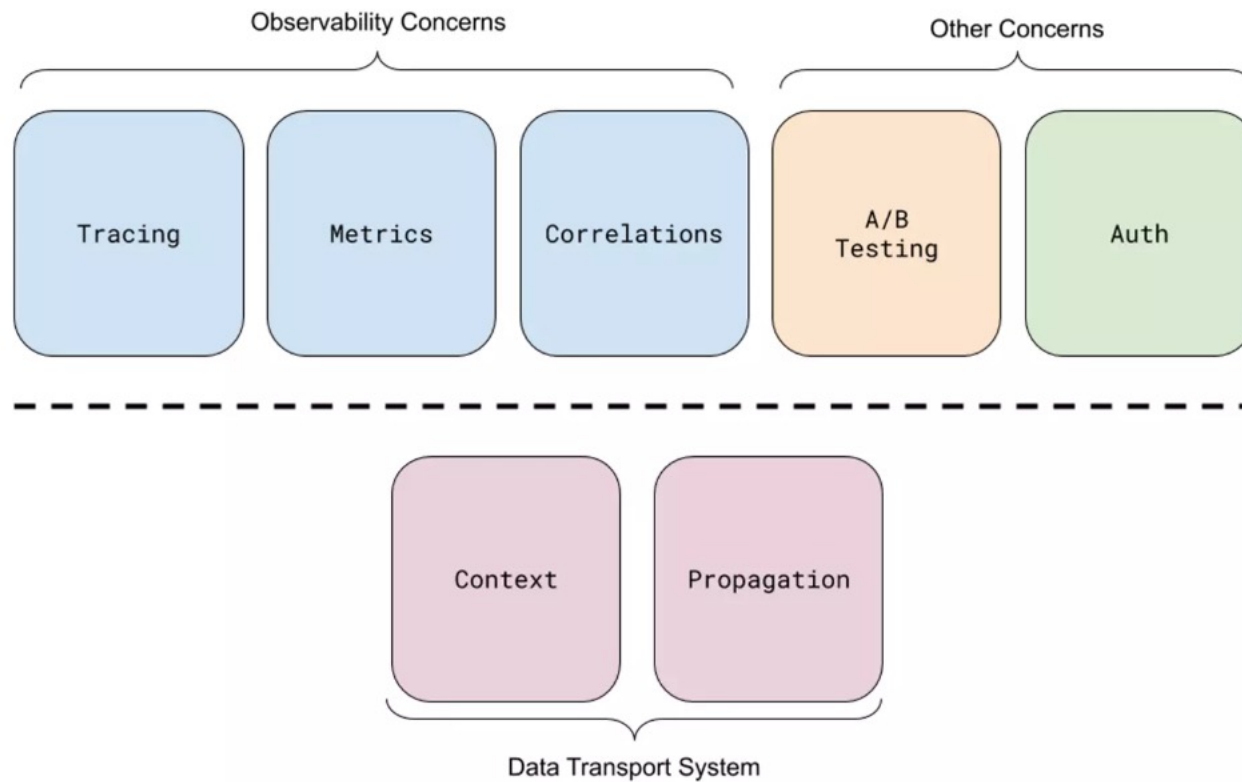
Follow the transaction

- At scale, it is impossible to find the relevant data. **Too much noise.**
- To investigate an issue, events and metrics must first be **contextualized**.
 - Grouped by transaction
 - Grouped by resource
 - Grounded by product attributes
- Adding this context is **a lot of work!**
- **OpenTelemetry does it for you.**

Single Transaction



Context propagation is fundamental!





OpenTelemetry Java Setup

Requirements

- Java 7+

Steps

- Install OpenTelemetry
- Install Instrumentation Adapters
- Configure the SDK
- Decorate your application code

Java Auto Installation



README: <https://github.com/open-telemetry/opentelemetry-auto-instr-java>

```
java -javaagent:path/to/opentelemetry-auto-0.2.0.jar \  
  -Dota.exporter.jar=path/to/opentelemetry-auto-exporters-otlp-0.2.0.jar \  
  -Dota.exporter.otlp.endpoint=localhost:55678 \  
  -jar myapp.jar
```

Library Adapters

Find out what's currently available by checking the registry: <https://opentelemetry.io/registry>

- [Akka HTTP](#)
- [Apache HttpAsyncClient](#)
- [Apache HttpClient](#)
- [AWS SDK](#)
- [Cassandra Driver](#)
- [Couchbase Client](#)
- [Dropwizard Views](#)
- [Elasticsearch API](#)
- [Elasticsearch REST Client](#)
- [Finatra](#)
- [Geode Client](#)
- [Google HTTP Client](#)
- [Grizzly](#)
- [gRPC](#)
- [Hibernate](#)
- [URLConnection](#)
- [Hystrix](#)
- [java.util.logging](#)
- [JAX-RS](#)
- [JDBC](#)
- [Jedis](#)
- [Jetty](#)
- [JMS](#)
- [JSP](#)
- [Kafka](#)
- [Lettuce](#)
- [Log4j](#)
- [Logback](#)
- [MongoDB Drivers](#)
- [Netty](#)
- [OkHttp](#)
- [Play](#)
- [Play WS](#)
- [Project Reactor](#)
- [RabbitMQ Client](#)
- [Ratpack](#)
- [RMI](#)
- [RxJava](#)
- [Servlet](#)
- [Spark Web Framework](#)
- [Spring Data](#)
- [Spring Scheduling](#)
- [Spring Servlet MVC](#)
- [Spring Webflux](#)
- [Spymemcached](#)
- [Twilio](#)

SDK Configuration

Propagator Options

- W3C
- B3
- Lightstep (deprecated)

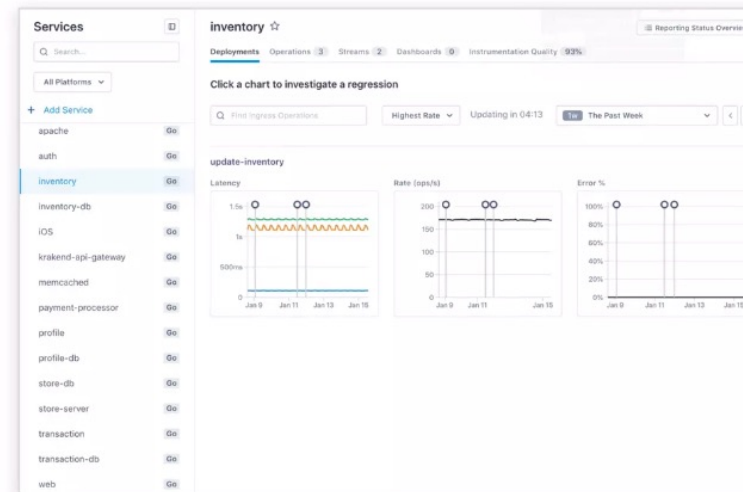
Exporter Options

- Lightstep
- OTLP
- Jaeger
- Console (debug)

```
Span span =  
tracer.spanBuilder("/resource/path").setSpanKind(Span.Kind.CLIENT).startSpan();  
  
span.setAttribute("http.method", "GET");  
span.setAttribute("http.url", url.toString());  
  
Map<String, AttributeValue> eventAttributes = new HashMap<>();  
eventAttributes.put("key", AttributeValue.stringAttributeValue("value"));  
eventAttributes.put("result", AttributeValue.longAttributeValue(0L));  
span.addEvent("Event Name", eventAttributes);
```

The right observability solution

- Easily be able to see and understand your highly distributed environment
- Immediately pinpoint performance regression
- Enable developers to ship with confidence
- Enable easy sharing of information
- Keep context
- Won't miss an insight



Observability Today

Enterprise Applications

- Only instrumented with logging during initial development.
 - Logging oriented toward development, not operations
- Metrics and tracing only added later if at all as a separate project.
 - Each team creates their own system using familiar tools
 - Or enterprise commits to a specific APM vendor
- Logs, metrics and traces are never connected.

First Generation Observability Platforms



Search logs in ELK,
Lack context



Kibana



Elasticsearch

Homegrown tracing per
app mainly accessible by
developers



Customer experience
metrics



Grafana



Prometheus

Low-level metrics
and alerts

zenoss



OpenCensus + OpenTracing = OpenTelemetry



- OpenTracing:
 - Provides APIs and instrumentation for distributed tracing
- OpenCensus:
 - Provides APIs and instrumentation that allow you to collect application metrics and distributed tracing.
- OpenTelemetry:
 - An effort to combine distributed tracing, metrics and logging into a single set of system components and language-specific libraries.

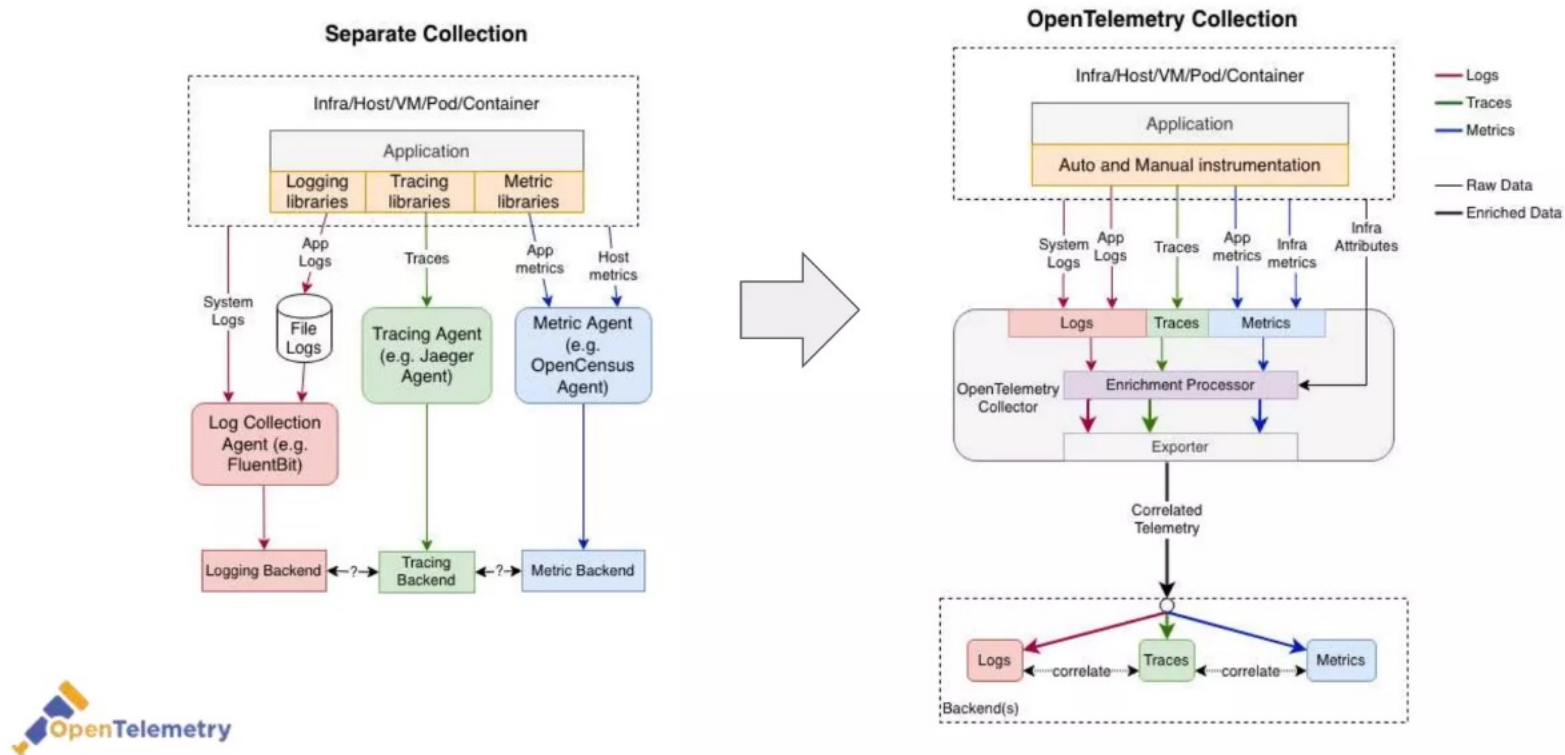


OpenTelemetry Project

- Specification
 - API (for application developers)
 - SDK Implementations
 - Transport Protocol (Protobuf)
- Collector (middleware)
- SDK's (various stages of maturity)
 - C++
 - C# (Auto-instrument/Manual)
 - Erlang
 - Go
 - JavaScript (Browser/Node)
 - Java (Auto-instrument/Manual)
 - Android compatibility
 - PHP
 - Python (Auto-instrument/Manual)
 - Ruby
 - Rust
 - Swift



From Observability 1.0 to 2.0



OpenTelemetry Collector



- Offers a vendor-agnostic implementation on how to receive, process and export telemetry data.
- Removes the need to run, operate and maintain multiple agents/collectors.
- Support open-source telemetry data formats (e.g. OTLP, Jaeger, Prometheus, etc.) sending to multiple open-source or commercial back-ends.

Collector Concepts

- Telemetry data processing pipelines
 - Per pipeline: Receiver(s) -> Processors -> Exporter(s)
 - Currently only single telemetry type pipelines supported
- Extensions
 - Supporting functionality
 - Core collector extensions
 - health_check - HTTP endpoint for load balancer or k8s controller
 - zpages - Internal processing metrics and traces accessible via HTTP
 - pprof - Performance profiler enables the golang net/http/pprof endpoint

Collector Bundled Receivers

Traces

- Jaeger
 - Compact Thrift, Binary Thrift, HTTP, gRPC
 - Sampling strategy configuration server
- Kafka
 - OTLP, Jaeger, Zipkin data structures
- OpenCensus
- OTLP (OpenTelemetry Protocol)
 - gRPC, HTTP
- Zipkin
 - v1, v1 Thrift, v2, v2 Protobuf



Metrics

- Host metrics scrapper
 - cpu, disk, load, filesystem, memory, network, processes, swap, process
- Kafka
 - OTLP

Collector Contrib Receivers

Traces

- AWS X-Ray
- SignalFX APM v1

Logs

- SignalFX (Events)
- Stanza

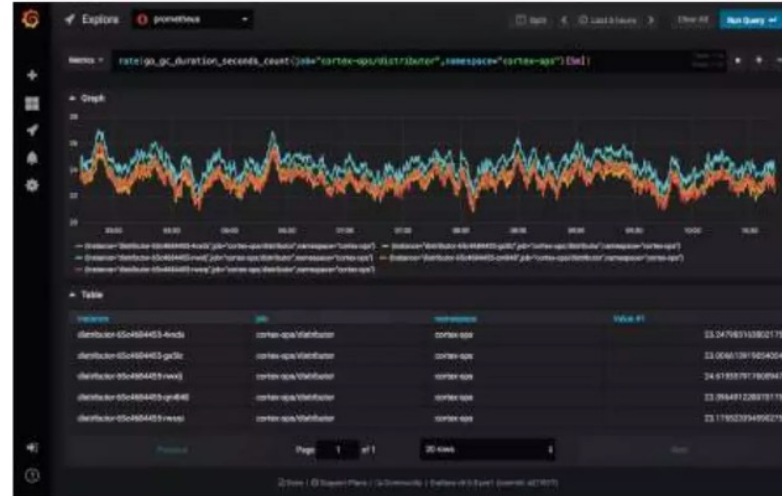


Metrics

- AWS ECS Container
- Carbon
- CollectD (JSON only)
- Docker Stats
- Kubernetes Cluster
- Kubernetes Kubelet
- Prometheus Exporters
- Redis INFO
- SignalFX
- Splunk HEC
- StatsD
- Wavefront

Metrics Concepts

- Gauges
 - Instantaneous point-in-time value (e.g. CPU utilization)
- Cumulative counters
 - Cumulative sums of data since process start (e.g. request counts)
- Cumulative histogram
 - Grouped counters for a range of buckets (e.g. 0-10ms, 11-20ms)
- Rates
 - The derivative of a counter, typically. (e.g. requests per second)



Basic Observability Metrics Methods

- **USE** - Utilization, Saturation, and Errors
 - Resource-scoped
- **RED** - Rate, Errors, and Duration
 - Request-scoped

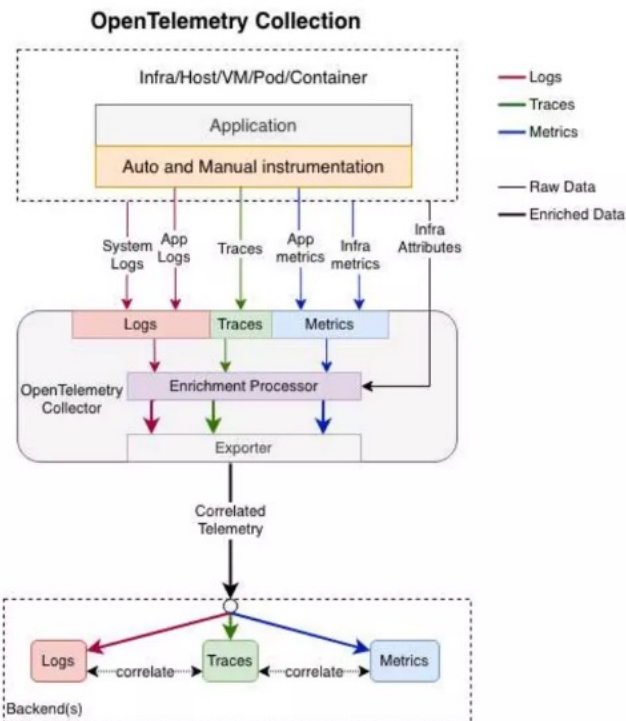
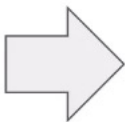
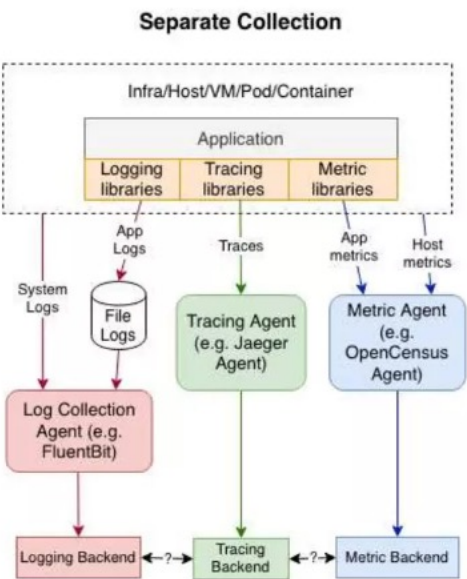
Observability 1.0 Limitations

- Data ends up in 3 different datastores.
- Different types of data not correlated with each other.
- Observability is not necessarily insight.

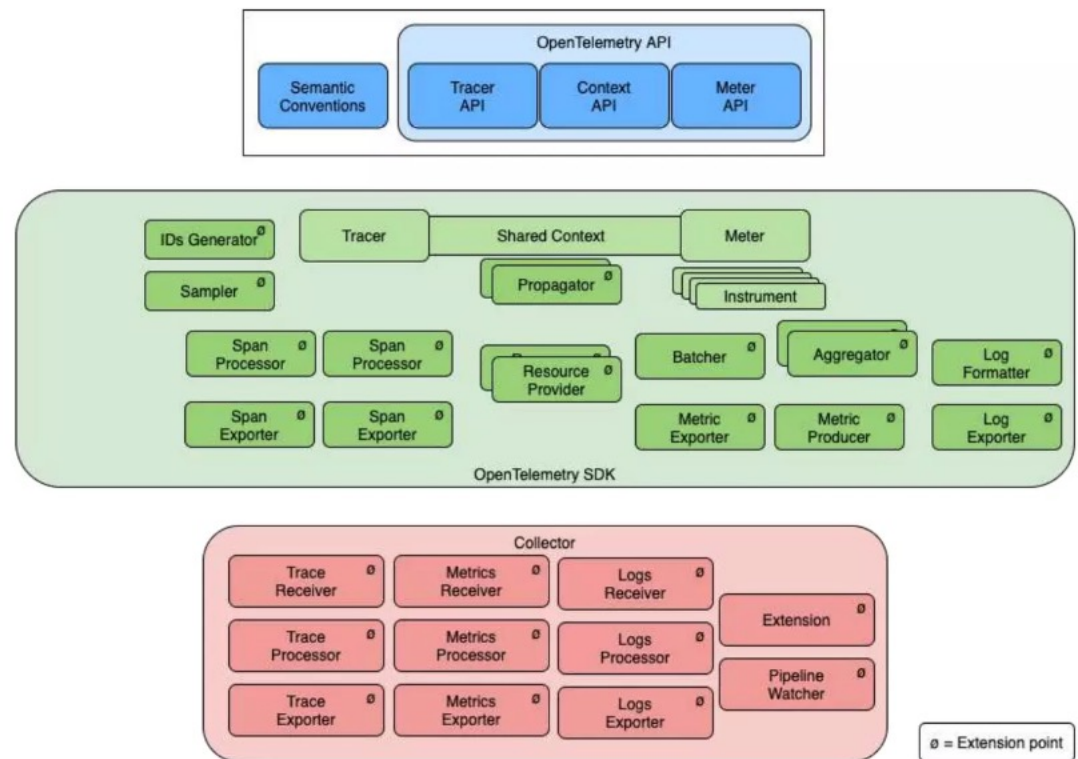
Operational Complexity Growth

	2010	2020
Circuit Breaker	Homegrown w/ 3 configs	Resilience4J w/ 14 configs
Retries	End user clicks submit again	Resilience4J w/ 7 configs
Health Check	HTTP server and DB are live	Kubernetes liveness, readiness, and startup probes with 5 timing configs per probe
Alerts	Unread count on circuit breaker opened email folder	???

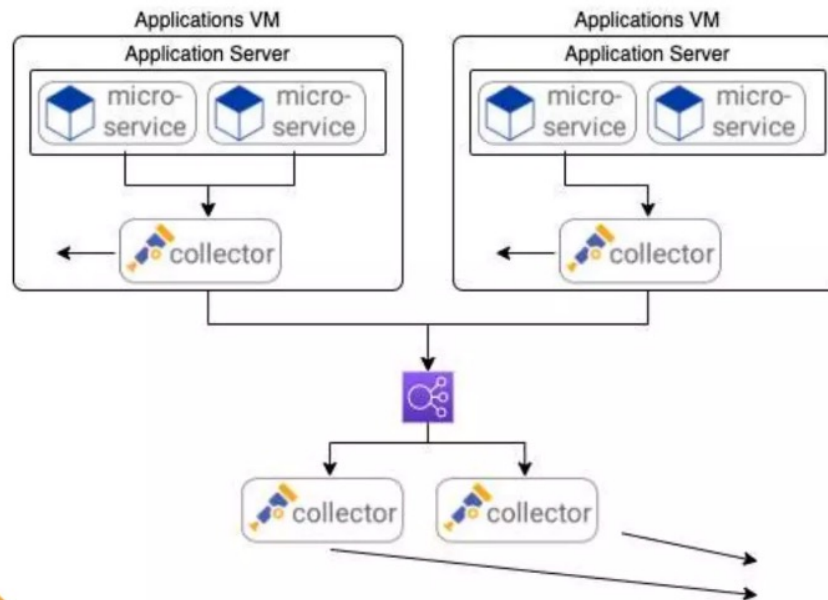
From Observability 1.0 to 2.0



Architecture



Application Server on VM Deployment



```
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, ...]
      exporters: [otlp]
    metrics:
      receivers: [statsd, otlp]
      processors: [memory_limiter, resource, ...]
      exporters: [otlp]
```

```
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch, queued_retry]
      exporters: [commercial...]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch, queued_retry]
      exporters: [commercial...]
```

Already Instrumented Applications

- OpenCensus
 - Application -> Collector (OpenCensus receiver) -> Backend
- OpenTracing
 - Application w/OT + OpenTracing shim + SDK -> Collector (OTLP receiver) -> Backend
- Spring Boot
 - Application w/Micrometer -> Collector (Prometheus receiver) -> Backend
 - Application w/Spring Cloud Sleuth -> Collector (Zipkin receiver) -> Backend
- AWS
 - Application w/X-Ray SDK -> Collector (X-Ray receiver) -> Backend(s)

Non-instrumented Applications



- Java
 - Launch with OpenTelemetry Java Agent (support for 61 widely-used frameworks and libraries)
- Javascript/Typescript
 - Add handlers/wrappers at key places or Node auto-instrumentation
- Microservice in any language
 - Deploy Envoy proxy as sidecar
- Infrastructure
 - Move to public cloud. AWS, Azure, GCP are all incorporating the OpenTelemetry collector in their infrastructure



Configuring Dependencies - Java

```
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-api</artifactId>
  <version>0.8.0</version>
</dependency>
...

implementation 'io.opentelemetry:opentelemetry-api:0.8.0'
implementation 'io.opentelemetry:opentelemetry-extensions-{extension}:0.8.0'
implementation 'io.opentelemetry:opentelemetry-sdk:0.8.0'
// OR implementation 'io.opentelemetry:opentelemetry-sdk-{tracing|metrics|baggage}:0.8.0'
implementation 'io.opentelemetry:opentelemetry-exporters-{exporter}:0.8.0'
implementation 'io.opentelemetry:opentelemetry-sdk-extensions-{extension}:0.8.0'
```

Baggage API Concepts



- Provides context and information to trace, metrics and logs telemetry
 - Set of name/value pairs
 - Contains info such as user, session, A/B testing parameters
 - Usually modeled as an immutable [Context](#) with each attribute add creating a new Context
 - Can be cleared if sending to untrusted process
- May be propagated across process boundaries
 - Using the header [otcorrelations](#) until W3C Baggage specification reaches recommendation status
- [BaggageManager](#) creates new [Baggage](#) objects based on current context

Baggage Example - Java Servlet

```
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
    throws IOException, ServletException {
    HttpServletRequest req = (HttpServletRequest) request;
    BaggageManager manager = OpenTelemetry.getBaggageManager();
    Baggage baggage = manager.baggageBuilder().setParent(manager.getCurrentBaggage())
        .put("enduser.id", req.getUserPrincipal().getName(),
            EntryMetadata.create(EntryTtl.UNLIMITED_PROPAGATION))
        .put("app.abflag", req.getHeader("X-ABTest-Flag"), EntryMetadata.create(EntryTtl.NO_PROPAGATION))
        .build();
    try (Scope scope = manager.withContext(baggage)) {
        chain.doFilter(request, response);
    }
}
```

Tracing API Concepts

- [TracerProvider](#) is the entry point of the API. It provides access to Tracers.
 - Stateful object holding configuration with a global provider and possibly additional ones.
- [Tracer](#) is the class responsible for creating Spans.
 - Named and optionally versioned with each instrumentation library using values guaranteed to be globally-unique.
 - Delegates getting active Span and marking a given Span as active to the [Context](#).
- [Span](#) is the API to trace an operation.
 - Immutable [SpanContext](#) represents the serialized and propagated portion of a Span.

Span Fields

- **Name** - Concisely identifies the work
- **SpanContext** - Immutable unique id
 - **TraceId** (16-bytes) globally-unique
 - **SpanId** (8-bytes) unique-within-trace
 - **TraceFlags** - Currently only sampled
 - **TraceState** - Vendor-specific info
- **ParentSpan** - Span which spawned this span
- **SpanKind** - Relationship to other spans
 - [CLIENT | SERVER | PRODUCER | CONSUMER | INTERNAL]
- **StartTimestamp** - Unix nanos
- **EndTimestamp** - Unix nanos
- **Attributes** collection
 - Key-value pairs describing the operational context
 - OTel semantic conventions define common keys
- **Links** collection
 - Associated non-parent spans
- **Events** collection
 - Significant data points during span
- **Status** - Success or failure details

Span Attribute Examples

HTTP Server

http.method	PUT
http.scheme	https
http.server_name	api22.opentelemetry.io
net.host.port	443
http.target	/blog/posts/774356
http.client_ip	2001:506:71f0:16e::1
http.user_agent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.138 Safari/537.36



RDBMS Client

db.system	mysql
db.connection_string	Server=shopdb.example.com;Database=ShopDb;Uid=billing_user;TableCache=true;UseCompression=True;MinimumPoolSize=10;MaximumPoolSize=50;
net.host.ip	192.0.3.122
net.host.port	51306
net.peer.ip	192.0.2.12
net.peer.port	3306
db.statement	SELECT * FROM orders WHERE order_id = :oid



Tracing Code - Java

```
Tracer tracer = OpenTelemetry.getTracer("instrumentation-library-name", "semver:1.0.0");
Span span = tracer.spanBuilder("my span").startSpan();
span.setAttribute("http.method", "GET");
span.setAttribute("http.url", url.toString());
try (Scope scope = tracer.withSpan(span)) {
    // your use case
    Attributes eventAttributes = Attributes.of(
        "key", AttributeValue.stringAttributeValue("value"),
        "result", AttributeValue.longAttributeValue(0L));
    span.addEvent("End Computation", eventAttributes);
} catch (Throwable t) {
    Status status = Status.INTERNAL.withDescription(t.getMessage());
    span.setStatus(status);
} finally {
    span.end(); // closing the scope does not end the span, this has to be done manually
}
```

Metrics API Concepts

- **MeterProvider** is the entry point of the API. It provides access to Meters.
 - Stateful object holding configuration with a global provider and possibly additional ones.
- **Meter** is class responsible for creating and managing Instruments.
 - Named and optionally versioned with each instrumentation library using values guaranteed to be globally-unique.
 - Provides means of capturing a batch of measurements in atomic way.
- **Instrument** is a device for capturing raw measurements.
 - Identified by a LabelSet (set of key-value pairs)
 - Bound instruments have pre-associated set of labels
 - SDK provides means for aggregating measurements captured by an instrument before export.

Metric Instrument Types



Name	Instrument Kind	Function(argument)	Default Aggregation
Counter	Synchronous additive monotonic	Add(increment)	Sum
UpDownCounter	Synchronous additive	Add(increment)	Sum
ValueRecorder	Synchronous	Record(value)	MinMaxSumCount / DDSketch
SumObserver	Asynchronous additive monotonic	Observe(sum)	Sum
UpDownSumObserver	Asynchronous additive	Observe(sum)	Sum
ValueObserver	Asynchronous	Observe(value)	MinMaxSumCount / DDSketch

Metrics SDK Concepts



- [Aggregator](#) aggregates one or more measurements in a useful way.
- [Accumulator](#) receives metric events from the API, computes one Accumulation per active Instrument and Label Set pair.
- [Processor](#) receives Accumulations from the Accumulator, transforms into ExportRecordSet.
- [Exporter](#) receives ExportRecordSet, transforms into some protocol and sends it somewhere.

Metrics Code - Java



```
Meter meter = OpenTelemetry.getMeter("instrumentation-library-name", "semver:1.0.0");
// Build counter e.g. LongCounter
LongCounter counter = meter.longCounterBuilder("processed_jobs").setDescription("Processed jobs").setUnit("1").build();
// It is recommended that the API user keep a reference to a Bound Counter for the entire time or
// call unbind when no-longer needed.
BoundLongCounter someWorkCounter = counter.bind(Labels.of("Key", "SomeWork"));
// Record data
someWorkCounter.add(123);
// Alternatively, the user can use the unbounded counter and explicitly
// specify the labels set at call-time:
counter.add(123, Labels.of("Key", "SomeWork"));
// Build observer e.g. LongObserver
LongObserver observer = meter.observerLongBuilder("cpu_usage").setDescription("CPU Usage").setUnit("ms").build();
observer.setCallback(
    new LongObserver.Callback<LongObserver.ResultLongObserver>() {
        @Override
        public void update(ResultLongObserver result) {
            // long getCpuUsage()
            result.observe(getCpuUsage(), Labels.of("Key", "SomeWork"));
        }
    }
);
```



Logging Concepts

- No API, use existing libraries
- Log events enhanced with key-value pairs from SpanContext and Baggage by either SDK or Collector processor.

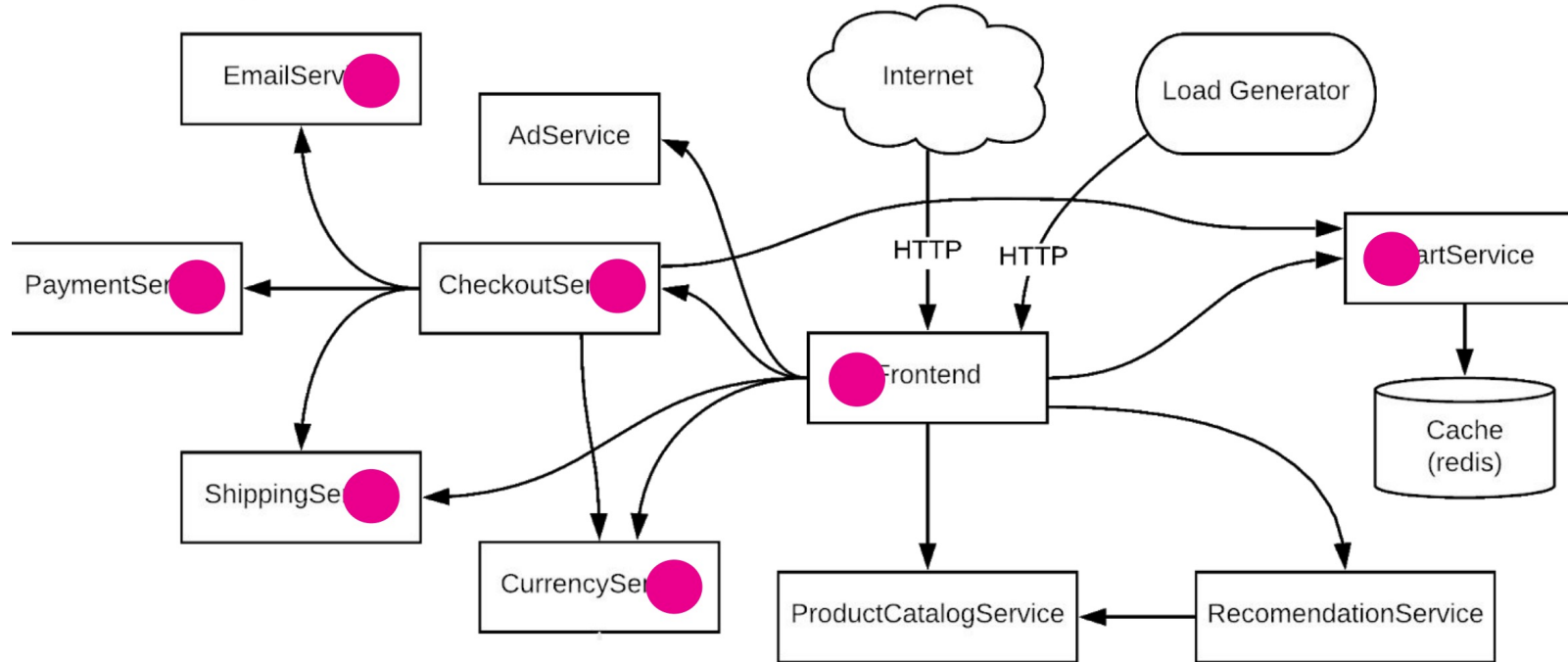
Exporting Data

Exporter Wiring

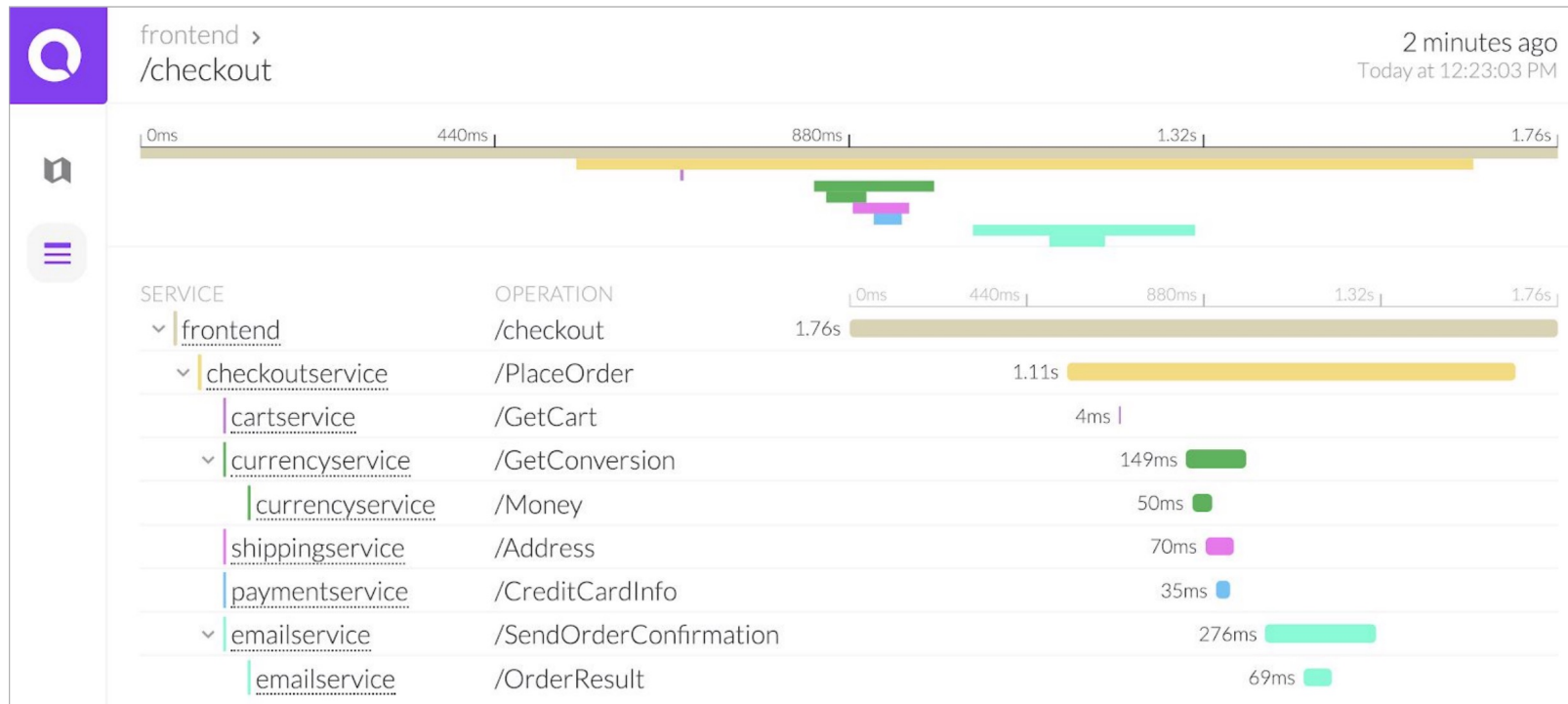


- Export to console logging or in-memory
 - Best solution for unit tests
- Export directly to observability frontend
 - Simple architecture but side processing in the application
- Export to otel-collector running as agent/sidecar which forwards to main otel-collector
 - Static configuration because exporter always pointed to localhost
- Export to otel-collector running as an aggregator
 - All telemetry processing done in isolation from user-facing functionality

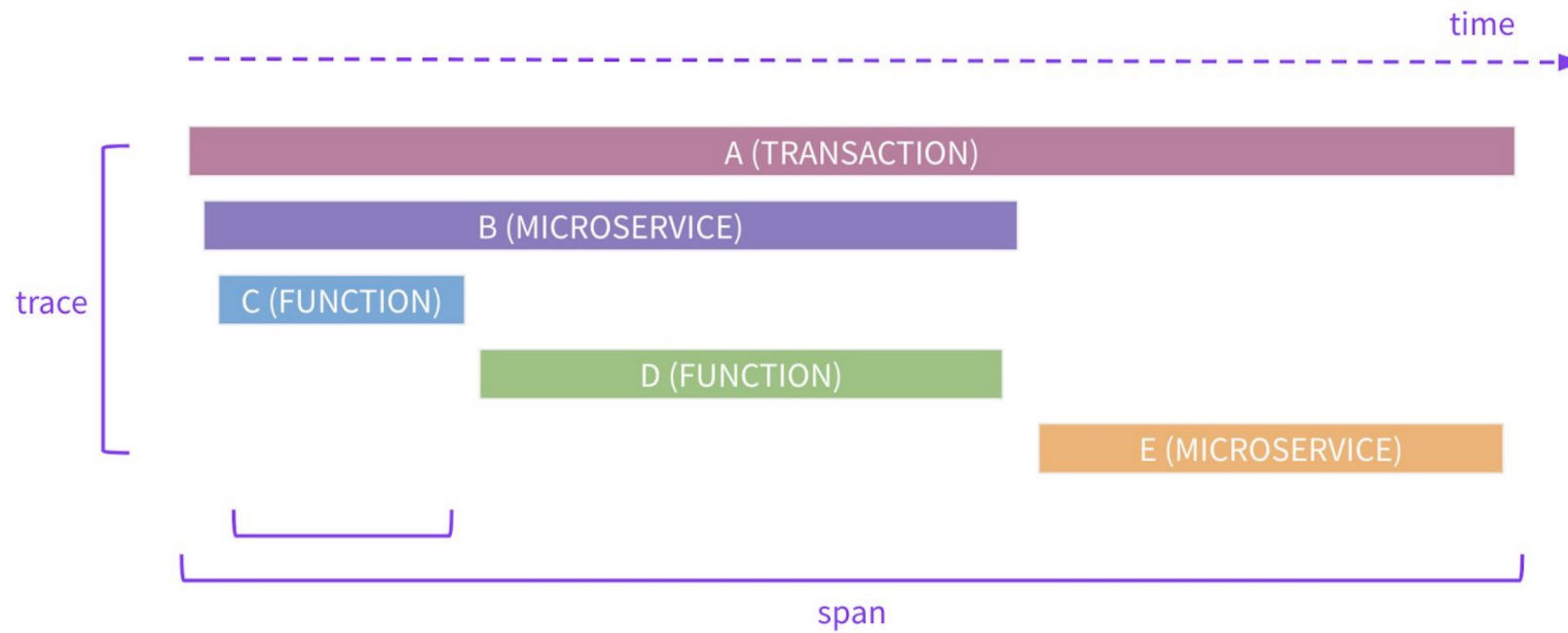
Example Microservice Architecture



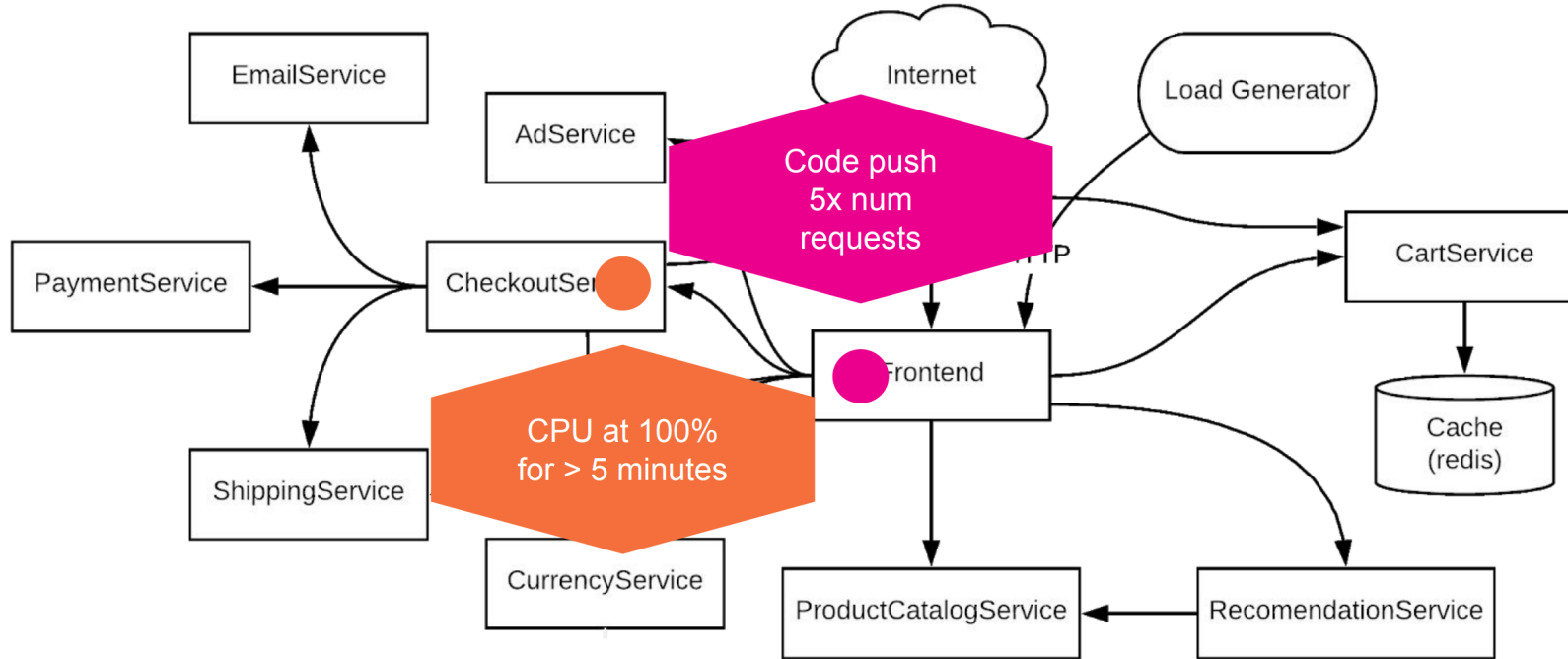
Distributed Trace Visualization



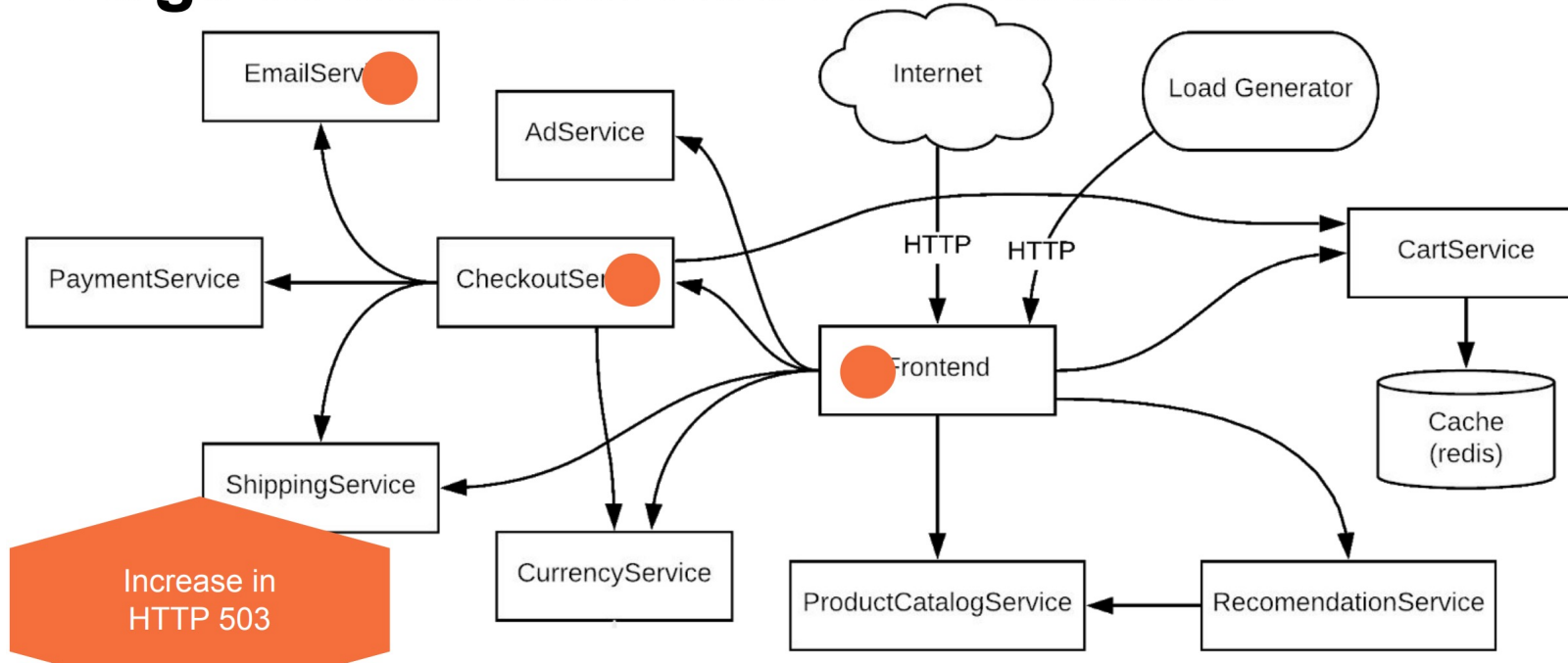
Distributed Trace Details

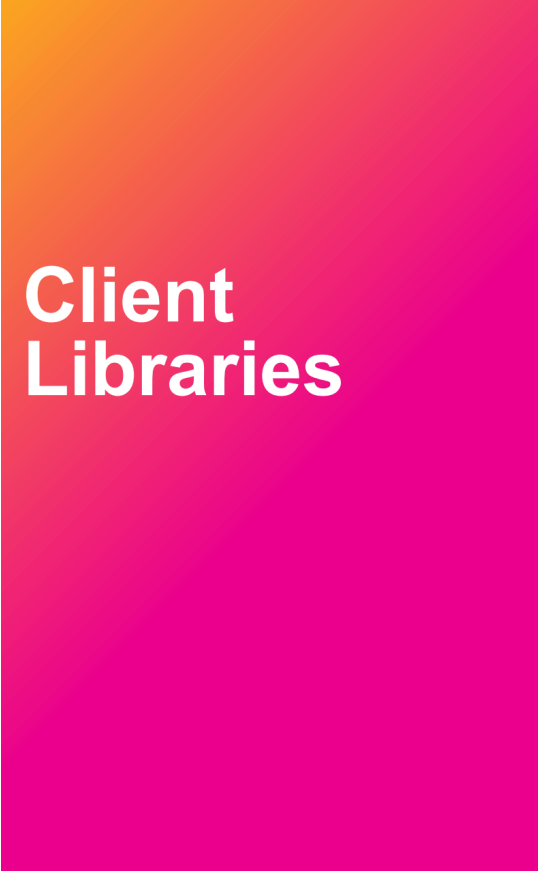


Metrics in Microservice Architecture



Logs in Microservice Architecture





Client Libraries

1. Add a client library dependency
 - Choose a context propagation format
 - Instantiate a tracer
 - Configure a destination
2. Enhance spans
 - Key/Value tags
 - Logs
3. Add more spans
 - Functional level instrumentation
 - Integrations (DBs, public cloud services, etc.)